Language

Автор сайта

Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

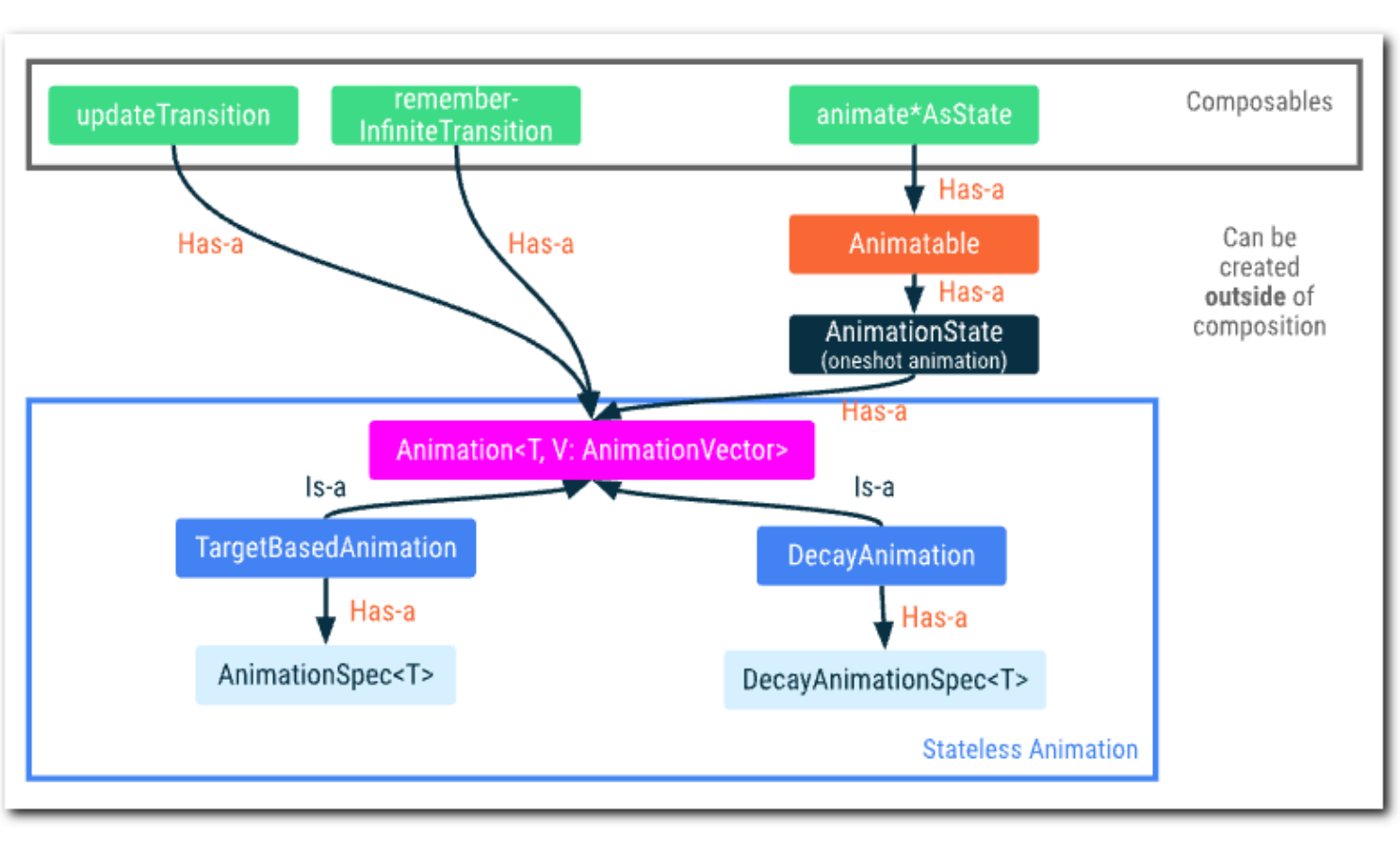
Урок 29. Анимация, основы.

18 августа 2023

В этом уроке обсуждаем основы анимации и разбираемся, как работает TargetBasedAnimation.

Начинаем серию уроков про анимацию. Официальная документация идет от верхнеуровневых компонентов к низкоуровневым.

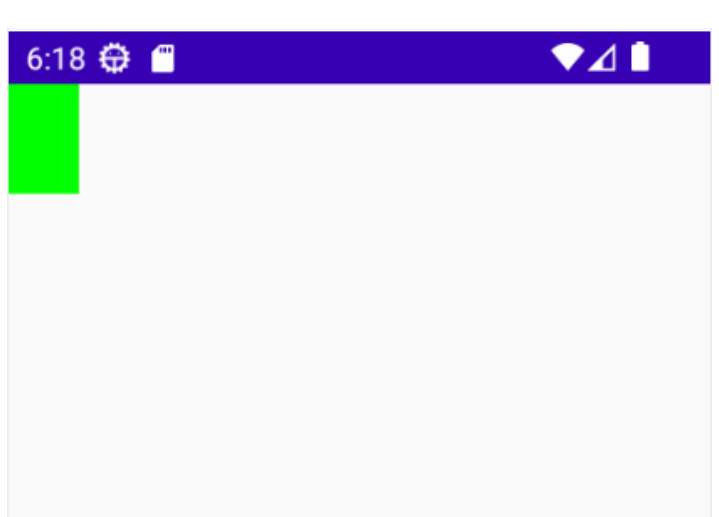
Т.е. сверху вниз на этом рисунке:



Мы же пойдем наоборот. Снизу вверх, чтобы лучше понимать как это все работает под капотом.

Анимация

Для начала давайте выясним, что из себя представляет анимация. Для примера можно взять простую анимацию - увеличение ширины UI элемента.



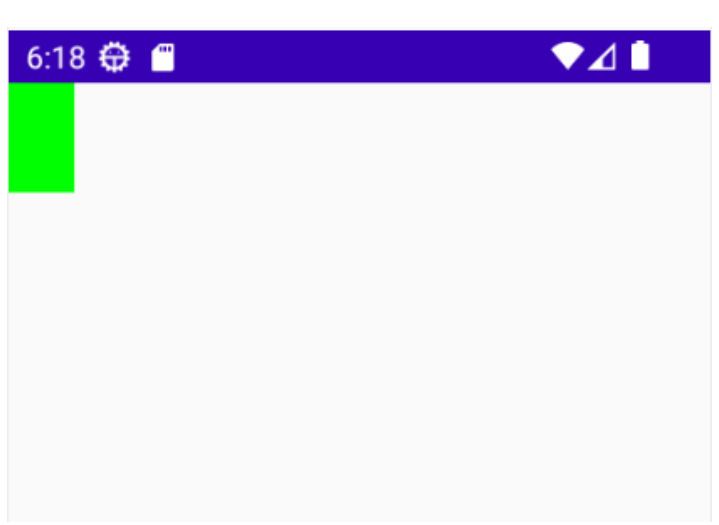
Никакой магии тут нет. Мы просто понемногу меняем ширину элемента, пока не достигнем нужного значения. Система постоянно перерисовывает UI элемент с учетом его новой ширины, и в итоге на экране возникает анимация.

Важно понимать способ, которым мы можем вычислять новое значение ширины в каждый следующий шаг. Первое, что приходит в голову - это просто брать текущее значение ширины и добавлять к нему какую то дельту. Т.е. что-то типа width = width + 1.dp. Но анимация использует другой подход. Она сама рассчитывает и предоставляет нам значение ширины на каждом этапе анимации. А мы просто помещаем это значение в width.

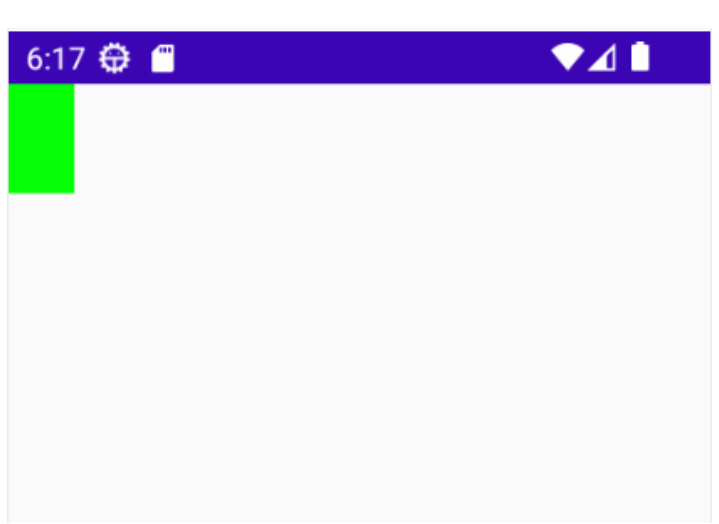
Чтобы анимация могла выполнить эти расчеты, от нас ей необходимо получить 3 компонента: начальное значение, конечное значение и спецификация.

Спецификация определяет, как именно будет меняться значение.

Это может быть постоянная скорость:



или медленный старт и быстрый финиш:



или пружина:

Подробнее про спецификации мы еще поговорим в отдельном уроке.

А пока перейдем к созданию простой анимации

TargetBasedAnimation

TargetBasedAnimation - это тот самый инструмент анимации, который будет рассчитывать для нас значение.

Давайте создадим один экземпляр и посмотрим на его входные параметры:

```
TargetBasedAnimation(
    animationSpec = tween(5000, easing = LinearEasing),
    typeConverter = Int.VectorConverter,
    initialValue = 0,
    targetValue = 100
)
```

Параметры initialValue, targetValue и animationSpec - это три компонента, которые я ранее упоминал: начальное значение, конечное значение и спецификация.

Спецификацию я выбрал одну их самых простых для понимания. Это постоянная скорость (без ускорений) с длительностью 5 секунд. TargetBasedAnimation использует эту спецификацию, чтобы рассчитать, как должно меняться значение от 0 (initialValue) до 100 (targetValue) в каждый момент времени. В итоге он получит, что, например, через 1 секунду значение будет - 20. Через 2,5 секунды - 50. Через 5 секунд - 100.

Мы выяснили, зачем нужны три из четырех параметров: animationSpec используется, чтобы рассчитать значение от initialValue до targetValue в каждый момент времени. Осталось понять, зачем нужен четвертый параметр - typeConverter.

Вектор

В нашем примере от спецификации требуется рассчитать, как меняется значение от 0 до 100. Это очень простой пример. Но он может быть сложнее.

Например, мы хотим анимировать не ширину элемента, а его позицию на экране. Для этого нам надо рассчитать как меняется значение Offset: от Offset(0f,0f) до Offset(100f, 200f).

Или еще более сложный пример - цвет. В нем 4 значения - alpha, red, green, blue. И спецификация должна просчитать все эти значения, чтобы сделать, например, анимацию перехода от желтого цвета к синему ;)

Конечно спецификация не настолько умна, чтобы уметь работать с любым типом данных: Int, Offset, Color, … Она в своей работе использует свой специальный тип - вектор. Поэтому от нас требуется предоставить конвертер, который сможет преобразовать наш тип в вектор (и наоборот).

В нашем примере TargetBasedAnimation использует Int конвертер, чтобы преобразовать начальные и конечные Int значения в векторы, и передает эти векторы в спецификацию. Спецификация все расчеты значения производит с векторами. Когда TargetBasedAnimation просит у спецификации вычислить значение в какой-либо момент времени (1 сек, 2.5 сек, 5 сек), спецификация возвращает ему вектор. Этот вектор прогоняется через конвертер и в итоге мы получаем Int значение.

Вектор представляет из себя просто набор Float значений, от одного до четырех:

```
class AnimationVector1D(initVal: Float) : AnimationVector() {...}
class AnimationVector2D(v1: Float, v2: Float) : AnimationVector() {...}
class AnimationVector3D(v1: Float, v2: Float, v3: Float) : AnimationVector() {...}
class AnimationVector4D(v1: Float, v2: Float, v3: Float, v4: Float) : AnimationVector() {...}
```

Например, Int будет преобразован в AnimationVector1D. Это можно увидеть в исходниках Int.VectorConverter:

```
private val IntToVector: TwoWayConverter<Int, AnimationVector1D> =
    TwoWayConverter({ AnimationVector1D(it.toFloat()) }, { it.value.toInt() })
```

В TwoWayConverter передаются две лямбды. Одна конвертирует Int в AnimationVector1D, другая - AnimationVector1D в Int.

Offset состоит из двух значений, поэтому будет преобразован в AnimationVector2D. В исходниках Offset.VectorConverter мы найдем такой код:

```
private val OffsetToVector: TwoWayConverter<Offset, AnimationVector2D> =
    TwoWayConverter(
        convertToVector = { AnimationVector2D(it.x, it.y) },
        convertFromVector = { Offset(it.v1, it.v2) }
    )
```

Ну а Color состоит из 4 значений, для него используется AnimationVector4D. Код исходника приводить не буду, там все сложно.

Таким образом TargetBasedAnimation избавляет нас от необходимости самим возиться с векторами. Мы работаем только с тем типом данных, который необходимо анимировать: Int, Offset, Color. А TargetBasedAnimation использует конвертер, чтобы преобразовать эти типы в вектор и обратно.

getValueFromNanos

Мы разобрались со всеми параметрами TargetBasedAnimation. Давайте посмотрим, как мы можем использовать этот объект.

У TargetBasedAnimation есть метод getValueFromNanos. Мы передаем туда время (в наносекундах), а в ответ получаем какое значение анимации будет в этот момент времени.

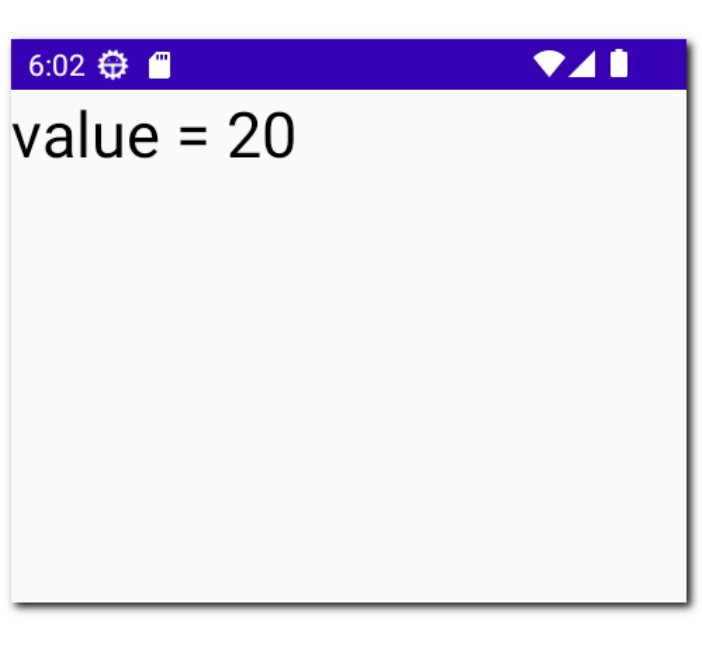
Простой пример:

```
@Composable
fun HomeScreen() {
    val animation = remember {
        TargetBasedAnimation(
            animationSpec = tween(5000, easing = LinearEasing),
            typeConverter = Int.VectorConverter,
            initialValue = 0,
            targetValue = 100
        )
    }

    val value = animation.getValueFromNanos(1_000_000_000)
    Text(text = "value = $value", fontSize = 30.sp)
}
```

Используем remember для создания TargetBasedAnimation, чтобы не создавать новую инстанцию при каждой рекомпозиции.

Вызываем метод getValueFromNanos и передаем туда одну секунду (1 * 10^9 наносекунд). А в ответ получаем 20.



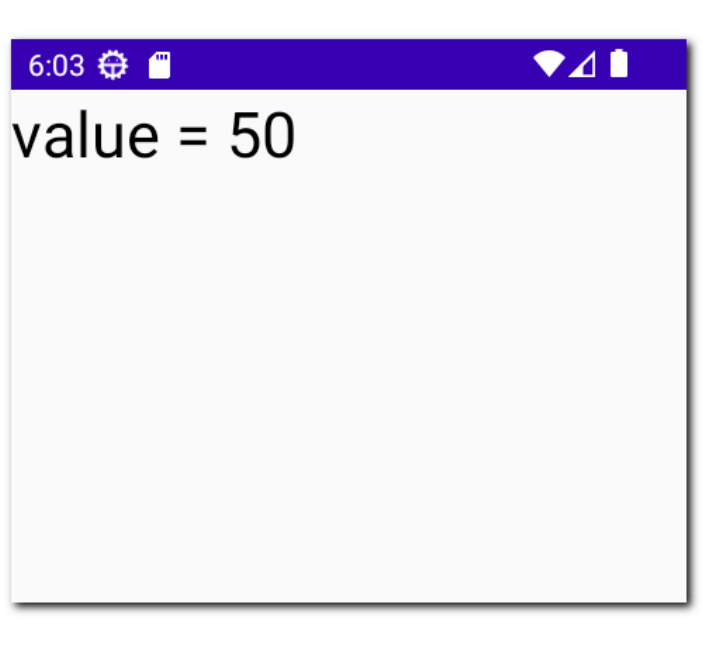
```
value = 20
```

Т.е. TargetBasedAnimation рассчитал, что если значение меняется от 0 до 100 за 5 секунд, то за одну секунду оно достигнет 20.

Попросим значение на момент 2.5 секунды:

```
val value = animation.getValueFromNanos(2_500_000_000)
```

Результат:



```
value = 50
```

Все верно, за половину времени, значение пройдет половину расстояния.

Чтобы анимация заработала, нам осталось создать цикл, который будет вести счет времени и постоянно вызывать метод getValueFromNanos, чтобы получить значение для анимации на текущий момент времени.

Это делается следующим образом:

```
val animationValueState = remember {
    mutableStateOf(0)
}

LaunchedEffect(key1 = Unit) {
    val startTime = withFrameNanos { it }
    var playTime = 0L
    while(playTime <= animation.durationNanos) {
        playTime = withFrameNanos { it } - startTime
        animationValueState.value = animation.getValueFromNanos(playTime)
    }
}
```

Для хранения значения создаем State. Туда цикл будет постить значения, а мы их оттуда будем брать и передавать в Text.

С помощью LaunchedEffect стартуем корутину, в которой запускаем цикл. В цикле используется метод withFrameNanos. Он возвращает время, прошедшее со старта девайса (SystemClock.uptimeMillis) в наносекундах. И он делает это строго один раз за кадр. А пока он ждет следующего кадра, корутина будет приостановлена. Таким образом в переменной playTime у нас идет счет времени от 0 до 5*10^9 наносекунд (animation.durationNanos). И каждый кадр мы передаем playTime в getValueFromNanos, чтобы получать значение для анимации в текущий момент времени. Это значение мы помещаем в animationValueState.

Осталось вывести его на экран, пока что просто в виде текста. Полный код примера выглядит так:

```
fun HomeScreen() {
    val animation = remember {
        TargetBasedAnimation(
            animationSpec = tween(5000, easing = LinearEasing),
            typeConverter = Int.VectorConverter,
            initialValue = 0,
            targetValue = 100
        )
    }

    val animationValueState = remember {
        mutableStateOf(0)
    }

    Text("value = ${animationValueState.value}", fontSize = 30.sp)

    LaunchedEffect(key1 = Unit) {
        val startTime = withFrameNanos { it }
        var playTime = 0L
        while(playTime <= animation.durationNanos) {
            playTime = withFrameNanos { it } - startTime
            animationValueState.value = animation.getValueFromNanos(playTime)
        }
    }
}
```

Результат:

```
value = 68
```

Попробуйте поменять продолжительность анимации или начальное/конечное значение, чтобы увидеть, как меняется скорость изменения значения.

Не пугайтесь количеству кода, который мы тут написали, чтобы получить такую простую анимацию. Мы сейчас кодим на низком уровне, чтобы понимать, как все это работает под капотом. Дальше будем подниматься на верхние уровни, и кода станет меньше.

А пока что наша цель - понять, что анимация это не магия, которая красиво меняет картинку на экране. Это цикл, который на каждом кадре использует спецификацию, чтобы вычислить новое значение. А мы это значение используем в UI, и получается анимация.

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **RE: Урок 29. Animation** — **Denis** 28.08.2023 23:27
Круто что разобрали внутренности, спс
Ответить

# **RE: Урок 29. Анимация, основы.** — **Azat** 01.10.2023 11:06
Спасибо за подробный урок!
Если не сложно, можете добавить, как сохранять состояние при перевороте экрана?
Ответить

> # **RE: Урок 29. Анимация, основы.** — **Dmitry Vinogradov** 13.10.2023 13:31
> Хороший вопрос. В эту сторону я пока не копал, но подозреваю, что простых решений тут нет. Только выносить весь мехнизм анимации в ViewModel и оттуда уже получать State с значением.
>
> Попробую посмотреть тему подробнее. Если найду что полезное, то напишу об этом в одном из уроков.
> Ответить

# **RE: Урок 29. Анимация, основы.** — **Alisher** 14.01.2024 15:07
Круто, спасибо!
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.